

# Codexion — Explicación visual y conceptual (español)

Documento pensado para que entiendas claramente los conceptos del subject: hilos, dongles, mutex, condvar, cooldown, scheduler (fifo/edf), monitor de burnout y requisitos de logs (incluido el umbral de 10 ms).

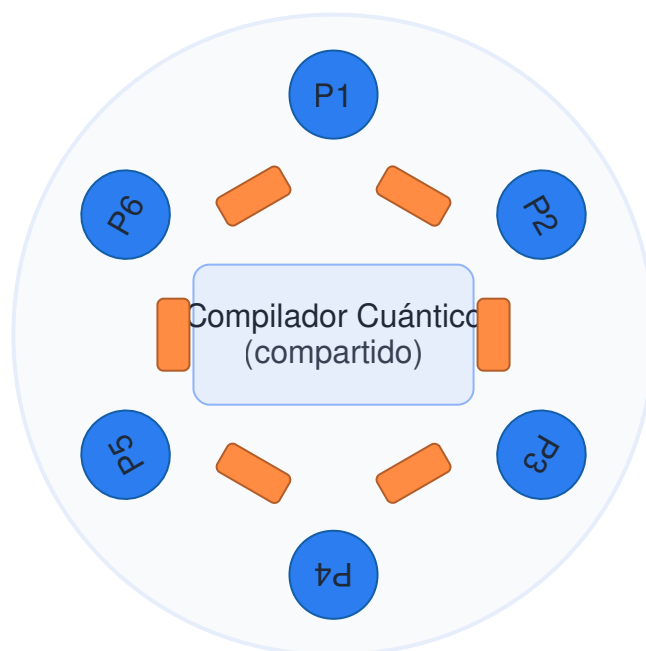
## 1. Visión general

Codexion es una simulación concurrente inspirada en el problema de los filósofos comensales. Cada **programador** es un hilo POSIX. Entre cada par de programadores hay un **dongle**. Para compilar, cada programador necesita tomar dos dongles simultáneamente (el de su izquierda y el de su derecha).

La simulación termina cuando uno se *agota* (burned out) por no empezar a compilar dentro de `time_to_burnout` ms desde su última compilación (o desde el inicio), o bien cuando todos han compilado al menos `number_of_compiles_required` veces.

## 2. Diagrama: Mesa circular (programadores y dongles)

Mesa circular: programadores y dongles



Observa la regla de la vecindad: el programador  $N$  tiene dongle a su izquierda ( $N - 1$ ) y a su derecha ( $N + 1$ ). Para compilar necesita ambos. Si  $N = 1$ , su izquierda es  $N = \text{number\_of\_coders}$  (la mesa es circular).

### 3. Parámetros: qué significan exactamente

| Argumento                                | Significado práctico y cómo afecta a la simulación                                                                                                                                                                                           |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>number_of_coders</code>            | Número de hilos (y número de dongles).<br>Determina la topología circular. Ejemplo: 5 -> 5 programadores y 5 dongles.                                                                                                                        |
| <code>time_to_burnout (ms)</code>        | Tiempo máximo que puede pasar sin que el programador comience a compilar desde su última compilación (o inicio). Si no comienza a compilar antes de ese tiempo, el monitor lo marca <b>burned out</b> . Es la <i>deadline</i> usada por EDF. |
| <code>time_to_compile (ms)</code>        | Duración de la acción "compilar". Mientras compila, el hilo debe sostener dos dongles.<br>Reduce disponibilidad de recursos.                                                                                                                 |
| <code>time_to_debug (ms)</code>          | Tiempo que pasa en la fase de debugging tras compilar; libera dongles al empezar debugging (según el enunciado las devuelve cuando termina de compilar y pasa a debugging).                                                                  |
| <code>time_to_refactor (ms)</code>       | Tiempo en refactorizar tras debugging. Al finalizar, el hilo intentará inmediatamente volver a adquirir dongles para compilar de nuevo (por tanto puede volver a competir).                                                                  |
| <code>number_of_compiles_required</code> | Si todos han compilado al menos este número, la simulación termina con éxito.                                                                                                                                                                |
| <code>dongle_cooldown (ms)</code>        | Después de que un dongle se libera, debe esperar este tiempo antes de poder asignarse otra vez.<br>Esto modela retraso físico o tiempo de desconexión y evita que se reasigne instantáneamente.                                              |
| <code>scheduler (fifo edf)</code>        | Política para decidir a quién asignar un dongle cuando varias personas lo piden. <b>fifo</b> = orden de llegada; <b>edf</b> = quien tiene deadline más cercano (deadline = last_compile_start + time_to_burnout).                            |

**Validaciones:** todos los argumentos son obligatorios. Rechaza entradas inválidas (valores negativos, no enteros, scheduler distinto de fifo/edf).

## 4. Cómo se maneja la adquisición de dongles (concepto de exclusión mutua)

Cada dongle es un recurso único —su estado debe protegerse con un *mutex*. Además hay que gestionar colas para los hilos que esperan (condvars o una estructura de espera propia).

Aquí las estrategias comunes:

- **Mutex por dongle:** cada dongle tiene su propio `pthread_mutex_t` y un estado (libre / ocupado / cooling\_until).
- **Arbitro central:** en vez de que cada hilo intente coger dongles por su cuenta, puedes tener un componente central que procesa peticiones y asigna 2 dongles de forma atómica (evita deadlocks pero añade complejidad de coordinación).
- **Condvars:** para hacer que hilos esperen y sean despertados cuando el dongle esté disponible (usar `pthread_cond_wait` o `pthread_cond_timedwait` para wakes temporizados).

**Importante:** Si permites que cada hilo intente tomar primero su izquierdo y luego su derecho sin control, se puede producir un deadlock (todos toman su izquierdo y esperan por el derecho). Por eso en problemas estilo filósofos se emplean estrategias como:

- Tomar ambos dongles de forma atómica mediante el árbitro.
- Imponer un orden global para adquirir recursos (por ejemplo, siempre coger primero el dongle con el índice menor).
- Permitir que sólo N-1 hilos intenten coger dongles simultáneamente (si hay N recursos), evitando el caso donde todos intentan al mismo tiempo.

## 5. Scheduler: FIFO vs EDF (en detalle)

### FIFO

Funcionamiento: cada dongle mantiene una cola FIFO. Cuando se libera, se concede al primero de la cola.

Ventajas: simple y predecible. Desventajas: no tiene en cuenta deadlines, por lo que alguien con riesgo de quemarse puede quedarse esperando.

### EDF (Earliest Deadline First)

Funcionamiento: cada petición lleva asociada una *deadline* calculada como `last_compile_start + time_to_burnout`. Cuando hay competencia, se atiende a la petición con deadline más cercana.

Ventajas: minimiza el riesgo de burnout si los parámetros permiten servir a los que están cerca de su límite. Requiere una estructura de prioridad (heap) para buscar rápidamente la petición con menor deadline.

Nota del enunciado: el programa debe garantizar que nadie se agote bajo EDF siempre que los parámetros sean viables. Eso no significa que EDF arregle casos imposibles (por ejemplo, tiempos demasiado pequeños para la carga dada).

## 6. Monitor de agotamiento (burnout) y la restricción de 10 ms

Debe existir un hilo monitor separado que vigile deadlines y detenga la simulación si alguien se agota. Requisitos clave:

- El monitor debe detectar el momento exacto en que la diferencia entre `now` y el inicio de la última compilación (o inicio) supera `time_to_burnout`.
- Al detectar el agotamiento, tiene que imprimir el log `timestamp_in_ms X burned out` dentro de 10 ms del momento real. Esto exige tiempos de wakeup precisos y una ruta de impresión prioritaria.

Consejos prácticos para cumplir los 10 ms:

1. Usa `gettimeofday()` para obtener timestamps (el enunciado lo permite y lo recomienda por simplicidad).
2. Evita que el monitor haga sleeps largos; usa `pthread_cond_timedwait` o temporizadores para despertarlo justo antes del siguiente deadline relevante.
3. Protege la salida con un mutex para que la impresión no se mezcle. Asegúrate de que el hilo que imprime pueda adquirir ese mutex rápidamente (no bloquearlo durante operaciones largas).
4. Minimiza trabajo dentro del path de detección e impresión (calcula el timestamp y escribe la línea; no hagas operaciones costosas antes de imprimir).

## 7. Logs: formato y serialización

Cualquier cambio de estado debe imprimirse exactamente con las líneas:

```
timestamp_in_ms X has taken a dongle
timestamp_in_ms X is compiling
timestamp_in_ms X is debugging
timestamp_in_ms X is refactoring
timestamp_in_ms X burned out
```

Notas:

- El `timestamp_in_ms` es el tiempo transcurrido desde el inicio de la simulación en milisegundos (usar `gettimeofday` y calcular la diferencia).
- La salida debe estar *serializada*: usa un mutex exclusivo para escritura en `stdout`/`printf`/`write` para evitar líneas mezcladas.
- Un mensaje que anuncie `burned out` no debe mostrarse más de 10 ms después del momento real.

## 8. Estructuras y primitivas permitidas

Funciones autorizadas (resumen):

```
pthread_create, pthread_join,  
pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, pthread_mutex_destroy,  
pthread_cond_init, pthread_cond_wait, pthread_cond_timedwait, pthread_cond_broadcast, p  
gettimeofday, usleep, write, malloc, free, printf, fprintf, strcmp, strlen, atoi, memse
```

Resumen de uso:

- `pthread_create` : crear hilos para cada programador y para el monitor.
- `pthread_mutex_*` : proteger estado compartido: dongles, contador de compilaciones, y la consola de logs.
- `pthread_cond_timedwait` : útil para implementar espera con timeout en las colas o para el monitor que espera hasta el próximo deadline.
- `gettimeofday` : para timestamps en ms.

## 9. Problemas habituales y cómo evitarlos

### Deadlock (interbloqueo)

Ocurre cuando todos los hilos toman un dongle y esperan el otro. Prevención:

- Tomar ambos dongles de forma atómica mediante un árbitro central que asigne pares de dongles.
- Imponer orden (ej. el hilo con índice par toma primero derecho, impar primero izquierdo) — pero esto puede causar otras complicaciones; el árbitro central es más sencillo conceptualmente.

### Inanición (starvation)

Alguien nunca recibe recursos por siempre. Con EDF esto se reduce si los parámetros permiten servir a quien tenga deadline cercano.

### Race conditions

Protégete usando mutexes alrededor de lecturas/escrituras de variables compartidas (estado de dongles, últimos tiempos de compilación, contador global de compilaciones).

## 10. Estrategia recomendada (pasos prácticos sin código)

1. Modela una estructura para cada dongle: contiene un `mutex`, un flag (libre/ocupado), y un timestamp `available_at` (para cooldown).
2. Implementa una cola de peticiones por dongle. Para FIFO puede ser una lista enlazada; para EDF usa un heap o priority queue ordenado por deadline.
3. Crea un hilo monitor que mantenga una lista (o heap) de deadlines por programador y que haga timed-waits hasta el próximo deadline. Cuando detecta expiración, imprime `burned out` y ordena terminar la simulación.
4. Cuando un programador quiere compilar, crea una petición con su id y deadline y la pone en las colas de los dongles que necesita. El árbitro (o la lógica de cada dongle) le asignará ambos dongles siguiendo la política elegida.
5. Al liberar un dongle, actualiza su `available_at = now + dongle_cooldown` y despierta a los waiters si corresponde (usar `pthread_cond_broadcast` tras actualizar estado).
6. Serializa logs con mutex. El monitor debe poder imprimir el `burned out` sin mucha demora.

## 11. Ejemplo de timeline (cómo leer los logs)

Ejemplo del enunciado (interpretación):

```
0 1 has taken a dongle
1 1 has taken a dongle
1 1 is compiling
201 1 is debugging
401 1 is refactoring
402 2 has taken a dongle
403 2 has taken a dongle
403 2 is compiling
603 2 is debugging
803 2 is refactoring
1204 3 burned out
```

Interpretación: los timestamps indican ms desde inicio. Observa cómo cada compiling está precedido por dos líneas de "has taken a dongle". Cuando alguien no alcanza su deadline, el log de "burned out" aparece (y debe imprimirse dentro de 10 ms del momento real).

## 12. Consejos finales para las pruebas

- Empieza con `number_of_coders` pequeño (2–3) y parámetros largos (por ejemplo `time_to_burnout = 2000 ms`) para ver la secuencia clara.
- Prueba casos límite: cooldown largo, `time_to_compile` muy corto, o `time_to_burnout` pequeño para observar burnout determinista.
- Comprueba fugas de memoria (valgrind o herramientas equivalentes) y manejo correcto de mutexes y condvars.
- Prioriza la determinación del monitor y la serialización de logs — son puntos críticos del evaluador.

Si quieres, puedo ahora:

1. Incluir un diagrama adicional que muestre una implementación con árbitro central (paso a paso).
2. Generar ejemplos de parámetros (casos de prueba) y explicar por qué cada caso es útil.
3. Preparar una lista de comprobación para la evaluación (qué deben validar los correctores) para usar en tu README o pruebas.